

TRANSCRIPT

Heavy real-service testing — nezha.local — iteration 1

Revision: 1 **Last modified:** 2026-06-16T13:10:00Z **Run id:** nezha_heavy_20260616T123646Z **Scope:** §11.4.69 sink-side real-service testing against the LIVE nezha stack (server-TLS <https://nezha.local:18443>, api <http://nezha.local:18080>, grpc nezha.local:50061, monitor :18090). No mocks — real LLMsVerifier bridge, real provider keys (provider llm-novita), real translations.

Endpoint-wiring FACT (Phase 1)

The Go heavy suites (test/integration, test/e2e, test/stress, test/distributed) are **self-contained by design** (§11.4.27): they use `httptest.NewServer` mock LLMsVerifier/OpenAI, MockTranslator, in-process gin engines, and dynamic ports (`test/utls/ports.go`, `localhost:0`). They read `NO_HELIX_*_URL/external endpoint env var` and cannot target nezha without rewriting their contract. The real-service surface is the HelixQA banks (`tests/banks/full-qa-{api,web,cli}.yaml`, hardcoded `localhost:8443 / :50051 / :8090`) which map 1:1 to the nezha ports. Heavy real testing was therefore done as direct sink-side probes against the live endpoints + running the Go suites at their own (deterministic) layer.

Bug #1 (FIXED) — LLM commentary contamination of translated text

Severity: Bug / release-critical (every translated paragraph polluted).

Discovery (sink-side, pre-fix): `POST :18443/api/v1/translate` returned the correct translation PLUS appended English meta-commentary, on EVERY provider response:

- en→es: El anciano y el mar eran uno.\n\nThis translation aims to preserve the poetic, almost mystical tone... (4 sentences of commentary)
- en→sr: Dobar dan, moj prijatelj.\n\n(Note: I've translated "Good morning"....)

- en→ru:
Быть или не быть, это вопрос.\n\n[Note: ...Shakespeare's Hamlet...]
- en→fr: Le savoir est le pouvoir.\n\nThis translation maintains the concise, aphoristic style...

Root cause (FACT, systematic-debugging Phase 1): the live translate path is handler → createTranslator → bridge BestTranslator → LLMTranslator.Translate → createTranslationPrompt (pkg/translator/llm/llm.go). The prompt builders end with "<Lang> translation:" and NEVER instruct the model to output only the translation; enhanceTranslation did no commentary stripping. A capable instruct model (llm-novita) helpfully appends an explanation, passed through verbatim. (The dead CreatePromptForLanguages in pkg/translator/universal.go had the same gap.)

Fix (defense-in-depth):

1. Prompt: added Output ONLY the translated text itself. Do NOT add any notes, explanations, commentary... to both createTranslationPrompt branches (R→S + generic) and CreatePromptForLanguages.
2. Response sanitization: new stripTranslationCommentary / isCommentaryBlock called from enhanceTranslation — drops trailing blank-line-separated blocks that are clearly commentary ((Note:)/[Note:] enclosures or English meta-phrase leads), conservatively (genuine multi-paragraph translations pass through untouched).

RED→GREEN proof: pkg/translator/llm/strip_commentary_test.go (§11.4.115 polarity, §11.4.135 standing guard) — fixtures are the LITERAL captured nezha responses. RED FAILED pre-fix (commentary leaked); GREEN post-fix; the *_DoesNotEatRealParagraphs guard proves no over-stripping.

Sink-side re-validation (post-fix, §11.4.130, new image 07e8a75614e6):

- en→es: "El viejo y el mar eran uno."
- en→fr: "Le savoir est le pouvoir."
- en→ru: "Быть или не быть – вот вопрос."
- en→de: "Der schnelle braune Fuchs springt über den faulen Hund."
- en→sr: "Dobra utra, moj prijatelj." All HTTP 200, ZERO commentary. Concurrent load: 12/12 parallel real translations HTTP 200, 0 commentary leaks, ~2s wall, no deadlock/error.

Bug #2 (FIXED) — create-default config skips env overrides (server crash-loop)

Severity: Bug / release-critical (fresh server deployment cannot start).

Discovery: after rebuilding the server image with Bug-#1 fix and recreating the container (fresh, no /app/config/config.json), the server crash-looped: Invalid configuration: JWT secret is required when authentication is enabled — despite JWT_SECRET correctly set in the container env.

Root cause (FACT, podman logs): `cmd/server/main.go loadOrCreateConfig`, when no `config.json` exists, returned a bare `config.DefaultConfig()` (`EnableAuth=true, JWTSecret=""`) WITHOUT applying environment overrides. The `env-loader` (`loadAPIKeysFromEnv`, which also loads `JWT_SECRET` + `LLMsVerifier`, `internal/config/config.go:314`) only ran on the `LoadConfig` (existing-file) path. The prior working run had a pre-existing `config.json` so it took the `LoadConfig` branch; a clean container hit the broken `create-default` branch.

Fix: added exported `(*Config).ApplyEnvOverrides()` (wraps `loadAPIKeysFromEnv`) and call it in `loadOrCreateConfig`'s `create-default` branch before `save/return`.

RED→GREEN proof: `internal/config/env_override_default_test.go` — RED FAILED (method undefined / JWT not applied to `DefaultConfig`); GREEN post-fix.

Sink-side re-validation: server reached Server started successfully! on the new image with the real 64-char JWT and bound :18443 (TLS /health 200 external).

Deploy path (§11.4.108 — fix reached the running artifact)

The `nezha` stack image (`helixtranslate:nezha`) builds on `nezha` from `Containerfile.nezha` (`COPY . .`). `deploy-stack` ships only `compose+artifacts`, NOT `source`, so fixed `source` was `rsynced` to `/home/milosvasic/helixtranslate/` (additive, no `--delete`; 9 `go.mod` replace targets present), then `podman-compose -f compose.nezha.yml up -d --build --no-deps server` with `.env.nezha` exported into the shell so `${SERVER_HOST_PORT}/${JWT_SECRET}` interpolate to the real values (host port 18443, 64-char JWT). New image digest `07e8a75614e6`; `server.Image` verified = new digest.

Tracked for iteration 2 (real findings, NOT yet fixed)

1. **Verifier REST endpoints 404 on live server** — `/api/v1/verified-models`, `/api/v1/verification-status`, `/api/v1/translate-with-verification` return 404 (HTQ-API-003/004/006 document them as critical). Cause: routes registered only if `cfg.LLMsVerifier.Enabled` (`cmd/server/main.go:140`); the `create-default` config has it disabled and no `LLMSVERIFIER_ENABLED` env set. Wiring gap — verifier routes dead on the `default-config` deployment.
2. **api-server /api/v1/health 503 "gRPC backend not ready"** (`grpc_connected: IDLE`) while `/api/v1/providers` (same backend) returns 200 — health check reports unhealthy on a lazily-IDLE gRPC conn that actually works. Health-check correctness bug.
3. **compose \${VAR} interpolation source** — `environment:` block defaults win over `env_file` and read from `shell/compose-.env`, not `.env.nezha`; deploy must export `.env.nezha` into the shell (or move these to `env_file-only`) so `ports/JWT` resolve. Deployment-robustness improvement.

4. gRPC real translate round-trip via grpcurl deferred (grpcurl absent on nezha
 - local; gRPC is async/file-job model). TCP :50061 reachable; api-server /api/v1/providers exercises the gRPC GetProviders path (200, real list).

Files

- 01_prefix_en_es_CONTAMINATED.json — pre-fix sink-side evidence (commentary).
- 02_postfix_en_es_CLEAN.json, 03_postfix_en_ru_CLEAN.json — post-fix clean.
- Full probe set: qa-results/nezha_heavy_test_20260616T123646Z/.